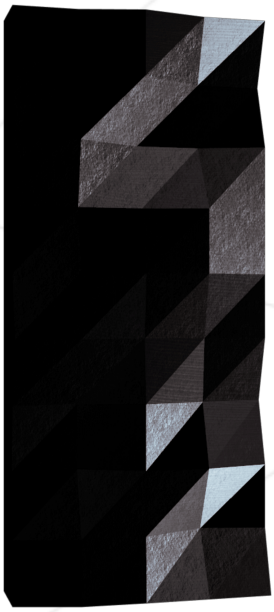

# El Códice

## Dominando los misterios de la importación en Python

*Resumen: Descubre el antiguo arte de la importación en Python a través de experimentos alquímicos. Domina los cuatro misterios sagrados: inicialización de paquetes, rutas de import, transmutaciones absolutas y relativas, y rotura de dependencias circulares.*

*Versión: 1.0*

# Índice general

**I. Prólogo** **2**

**II. Instrucciones sobre la IA** **3**

**III. Introducción** **6**

**IV. Instrucciones Generales** **8**
IV.1. Autorizado y Prohibido . . . . . 8
IV.2. Estructura del Laboratorio . . . . . 9

**V. Parte Obligatoria** **10**
V.1. Parte I: El Pergamino Sagrado . . . . . 10
V.2. Parte II: Transmutación por Importación . . . . . 13
V.3. Parte III: El Gran Debate de las Rutas . . . . . 15
V.4. Parte IV: Rompiendo la Maldición Circular . . . . . 17

**VI. Instrucciones de Entrega** **19**

# Capítulo I

# Prólogo

Te damos la bienvenida al Laboratorio Alquímico, aspirante!

Ya dominas los elementos básicos de Python: variables, funciones, clases y estructuras de datos. Ahora es el momento de aprender el antiguo arte de la transmutación de código a través del sistema de importación de módulos de Python.

En la Edad Media, las personas alquimistas buscaban transformar metales básicos en oro. Hoy, como alquimista de Python, aprenderás a transformar código disperso en fórmulas mágicas organizadas y reutilizables. Igual que las personas alquimistas de antaño tenían sus grimorios (libros de hechizos) organizados en capítulos y secciones, el código en Python debe organizarse en módulos y paquetes.

¡Pero cuidado! El camino de la alquimia está lleno de misterios:

- El Pergamino Sagrado (__init__.py): la puerta que transforma una simple carpeta en un paquete mágico.
- El Arte de la Transmutación por Import: invocar código desde módulos lejanos.
- El Gran Debate de las Rutas: imports absolutos frente a relativos, como elegir entre diferentes fórmulas alquímicas.
- La Maldición de las Dependencias Circulares: cuando los hechizos se referencian entre sí en un bucle infinito, ¡amenazando con desgarrar la estructura de tu código!

En este códice, construirás tu propio Laboratorio Alquímico: un paquete completo de Python que demuestra estos cuatro misterios sagrados. Cada experimento te enseñará un aspecto de la magia de la importación de módulos en Python.

Al final, dominarás el arte de organizar código, evitando las trampas en las que cayeron cantidad de aspirantes antes que tu.

# Capítulo II

# Instrucciones sobre la IA

## Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte con muchas tareas diferentes. Tómate el tiempo necesario para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyarte con tu trabajo. Sin embargo, siempre debes abordarlas con precaución y evaluar de forma crítica los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca podrás saber con total certeza si tu pregunta está bien formulada o si el contenido generado es el adecuado. Las personas que te rodean son tu recurso más valioso para ayudarte a evitar errores y puntos ciegos.

## Mensaje principal:

- Utiliza la IA para reducir las tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting, ya sea para programacion o para otros temas, que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar de forma eficiente y evitar los riesgos comunes, sesgos y problemas éticos.
- Sigue trabajando con tus compañeros para desarrollar tanto habilidades técnicas como habilidades transversales.
- Utiliza únicamente contenido generado por IA que entiendas completamente y del cual puedas responsabilizarte.

## Reglas para estudiantes:

- Debes tomarte el tiempo necesario para explorar las herramientas de IA y comprender cómo funcionan, para poder utilizarlas de manera ética y reducir los sesgos potenciales.
- Debes reflexionar sobre tu problema antes de dar instrucciones a la IA. Esto te ayuda a escribir preguntas, instrucciones o conjuntos de datos más claros, detalladas y relevantes utilizando un vocabulario preciso.

- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier contenido generado por la IA.
- Debes buscar siempre la revisión de otras personas, no te limites a confiar en tu propia validación.

## Resultados de esta etapa:

- Desarrollar habilidades de prompting tanto generales como de ámbito especifico.
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
- Seguir fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

## Comentarios y ejemplos:

- Ten en cuenta que la IA puede no tener la respuesta correcta porque esa respuesta no esté ni siquiera en Internet. Además, si te da soluciones incorrectas, intenta no insistir y busca ayuda entre las personas que te rodean. Vas a ahorrarte tiempo y vas a sumar en compresión.
- Vas a enfretarte con frecuencia a situaciones (como exámenes o evaluaciones) donde debes demostrar una comprensión real. Prepárate, sigue construyendo tanto tus habilidades técnicas como transversales.
- Explicar tu razonamiento y debatir con otras personas suele revelar lagunas en tu comprensión de un concepto. Prioriza el aprendizaje entre pares.
- Lo normal es que la herramienta de IA que utilices no conozca tu contexto específico (a menos que se lo indiques), así que te dará respuestas genéricas. Si buscas información más adecuada y más precisa en relación a tu entorno cercano, confía en el resto de estudiantes.
- Donde la IA tiende a generar la respuesta más probable, el resto de estudiantes puede proporcionar perspectivas alternativas y matices valiosos. Confía en la comunidad de 42 como un punto de control de calidad.

**✓ Buenas prácticas:**

> Le pregunto a la IA: "¿Cómo pruebo una función de ordenación?"Me da algunas ideas. Las pruebo y reviso los resultados con otra persona. Refinamos el enfoque de manera conjunta.

**✗ Mala práctica:**

> Le pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad. Suspendo mi proyecto.

**✓ Buenas prácticas:**

Utilizo la IA para ayudarme a diseñar un parser. Luego, reviso la lógica con otra persona. Encontramos dos errores y lo reescribimos juntos: mejor, más limpio y comprendiendo al 100

**✗ Mala práctica:**

Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y suspendo mi proyecto.

# Capítulo III

# Introducción

¡Te presentamos El Códice Alquímico!

Como aspirante a alquimista, descubrirás el sistema de importación de módulos de Python a través de experimentos mágicos prácticos. Cada experimento se construye sobre el anterior, creando un paquete completo de Laboratorio Alquímico que demuestra una organización profesional en Python.

**PRERREQUISITOS NECESARIOS**: Esta actividad asume un dominio sólido de los fundamentos de Python, incluyendo sintaxis, funciones, clases, gestión de errores, listas, diccionarios y estructuras de datos. Deberías poder escribir clases en Python, gestionar excepciones y trabajar con colecciones con comodidad antes de intentar esta actividad centrada en los imports. Sin estas bases, los conceptos de import serán difíciles de entender.

Céntrate en entender **cómo funcionan la importación de módulos**, no solo en conseguir que funcionen. ¡La esencia de la alquimia reside en entender cada paso!

Esta actividad trata sobre **organización de código** y **mecanismos de import**. Mantén tus fórmulas alquímicas simples (funciones básicas) para que puedas centrarte en los conceptos de import.

**GESTIÓN DE ERRORES**: Todas las funciones deben devolver strings. Cuando pruebes imports que puedan fallar (como acceder a funciones ocultas), utiliza bloques try/except y devuelve mensajes de error descriptivos en lugar de dejar que el programa se rompa.

# Capítulo IV

# Instrucciones Generales

## IV.1. Autorizado y Prohibido

**INGREDIENTES AUTORIZADOS**:

- Todos los módulos de la librería estándar de Python (datetime, math, os, sys, etc.).
- Crear tus propios módulos y paquetes.
- Todos los estilos de import: import, from...import, import...as.
- Crear archivos __init__.py (Pergaminos Sagrados).
- Imports absolutos y relativos.
- type hints (anotaciones de tipo).

**MAGIA OSCURA PROHIBIDA**:

- Librerías externas (nada de pip install).
- Usar eval() o exec() (transmutaciones peligrosas).
- Modificar sys.path directamente.
- Usar importlib para imports dinámicos.
- Algoritmos complejos (mantén tus hechizos simples).

## IV.2. Estructura del Laboratorio

Construirás tu Laboratorio Alquímico de forma progresiva a lo largo de cuatro partes:

- **Parte I**: El Pergamino Sagrado (misterio de __init__.py).
- **Parte II**: Transmutación por Import (dominio de from...import).
- **Parte III**: El Gran Debate de las Rutas (absoluto frente a relativo).
- **Parte IV**: Rompiendo la Maldición Circular (resolución de dependencias).

**Estructura Final del Laboratorio**:

- `alchemy/` - Tu paquete principal de laboratorio.
- `alchemy/elements.py` - Funciones de hechizos elementales básicos.
- `alchemy/potions.py` - Funciones de recetas de pociones avanzadas.
- `alchemy/transmutation/` - Paquete de hechizos de transformación.
- `alchemy/grimoire/` - Paquete de documentación de hechizos.

**IMPORTANTE**: Todas las funciones deben ser simples y devolver strings. Céntrate en los conceptos de import, no en lógica compleja. Gestiona los errores devolviendo mensajes de error descriptivos como strings.

# Capítulo V

# Parte Obligatoria

## V.1. Parte I: El Pergamino Sagrado

### Objetivo

Descubre el poder de ___init___.py: el pergamino sagrado que transforma carpetas ordinarias en paquetes mágicos de Python.

### Archivos que hay que crear para la Parte I

- `ft_sacred_scroll.py` - Script de demostración (en la raíz del repositorio).
- `alchemy/__init__.py` - El pergamino sagrado principal.
- `alchemy/elements.py` - Hechizos elementales básicos.

### Instrucciones

Crea tu primer paquete alquímico y aprende cómo ___init___.py controla qué magia está disponible para otras personas alquimistas.

**alchemy/elements.py debe contener**:

- `create_fire()` - Devuelve "Fire element created".
- `create_water()` - Devuelve "Water element created".
- `create_earth()` - Devuelve .Earth element created".
- `create_air()` - Devuelve .Air element created".

**alchemy/___init___.py debe contener exactamente**:

- `__version__ = "1.0.0"`.
- `__author__ = "Master Pythonicus"`.
- Importar y exponer ÚNICAMENTE `create_fire` y `create_water` desde elements.py.
- NO exponer `create_earth` y `create_air` (permanecen ocultas).
- Usar: `from .elements import create_fire, create_water`.

El archivo __init__.py controla la interfaz del paquete. Las funciones importadas aquí pasan a estar disponibles como alchemy.function_name. Las funciones no importadas permanecen ocultas y requieren acceso directo al módulo.

**Ejemplo de salida esperada**:
Comprueba tu pergamino sagrado:

```
$> python3 ft_sacred_scroll.py

=== Sacred Scroll Mastery ===

Testing direct module access:
alchemy.elements.create_fire(): Fire element created
alchemy.elements.create_water(): Water element created
alchemy.elements.create_earth(): Earth element created
alchemy.elements.create_air(): Air element created

Testing package-level access (controlled by __init__.py):
alchemy.create_fire(): Fire element created
alchemy.create_water(): Water element created
alchemy.create_earth(): AttributeError - not exposed
alchemy.create_air(): AttributeError - not exposed

Package metadata:
Version: 1.0.0
Author: Master Pythonicus
```

Tu ft_sacred_scroll.py debería demostrar tanto el acceso directo al módulo (alchemy.elements.function) como el acceso a nivel de paquete (alchemy.function). Gestiona las excepciones AttributeError de forma adecuada imprimiendo mensajes de error.

¿Cómo controla el Pergamino Sagrado (__init__.py) qué hechizos están disponibles para otras personas alquimistas? ¿Cuál es la diferencia entre lo que existe en un módulo y lo que el paquete expone?

## V.2. Parte II: Transmutación por Importación

### Objetivo

Domina el arte de from...import: invocar hechizos específicos desde grimorios lejanos sin traer el libro completo.

### Archivos que hay que crear para la Parte II

- `ft_import_transmutation.py` - Script de demostración (en la raíz del repositorio).
- `alchemy/potions.py` - Recetas de pociones avanzadas.

### Instrucciones

Expande tu laboratorio y aprende diferentes formas de invocar fórmulas mágicas.

**alchemy/potions.py debe contener exactamente**:

- `healing_potion()` - Devuelve "Healing potion brewed with [fire_result] and [water_result]".
- `strength_potion()` - Devuelve "Strength potion brewed with [earth_result] and [fire_result]".
- `invisibility_potion()` - Devuelve Ïnvisibility potion brewed with [air_result] and [water_result]".
- `wisdom_potion()` - Devuelve "Wisdom potion brewed with all elements: [all_four_results]".

`Cada función de poción debe importar y llamar a las funciones elementales necesarias, y después devolver un string que incluya los resultados elementales. Por ejemplo: from .elements import create_fire, create_water.`

**Tu demostración debería mostrar**:

- Diferentes estilos de import: `import alchemy.elements`.
- Imports específicos: `from alchemy.elements import create_fire`.
- Imports con alias: `from alchemy.potions import healing_potion as heal`.
- Imports múltiples: `from alchemy.elements import create_fire, create_water`.
- Cómo afecta cada estilo a tu código de forma diferente.

**Ejemplo de salida esperada**:
Comprueba tus métodos de transmutación:

```
$> python3 ft_import_transmutation.py

=== Import Transmutation Mastery ===

Method 1 - Full module import:
alchemy.elements.create_fire(): Fire element created

Method 2 - Specific function import:
create_water(): Water element created

Method 3 - Aliased import:
heal(): Healing potion brewed with Fire element created and Water element created

Method 4 - Multiple imports:
create_earth(): Earth element created
create_fire(): Fire element created
strength_potion(): Strength potion brewed with Earth element created and Fire element created

All import transmutation methods mastered!
```

Tu ft_import_transmutation.py debería demostrar claramente los cuatro estilos de importación. Cada método debe mostrar la sentencia de import utilizada y el resultado de la llamada a la función.

¿Cuáles son las ventajas y desventajas de cada método de transmutación por import? ¿Cuándo usarías import module frente a from module import function?

## V.3. Parte III: El Gran Debate de las Rutas

### Objetivo

Entiende el antiguo debate entre imports absolutos y relativos: dos caminos diferentes para llegar a la misma fórmula mágica.

### Archivos que hay que crear para la Parte III

- `ft_pathway_debate.py` - Script de demostración (en la raíz del repositorio).
- `alchemy/transmutation/__init__.py` - Inicializador del paquete transmutation.
- `alchemy/transmutation/basic.py` - Transmutaciones básicas.
- `alchemy/transmutation/advanced.py` - Transmutaciones avanzadas.

### Instrucciones

Crea una estructura de laboratorio compleja y aprende cuándo usar cada tipo de ruta.

**alchemy/transmutation/basic.py debe contener exactamente**:

- Import absoluto: `from alchemy.elements import create_fire, create_earth`.
- `lead_to_gold()` - Devuelve "Lead transmuted to gold using [fire_result]".
- `stone_to_gem()` - Devuelve "Stone transmuted to gem using [earth_result]".

**alchemy/transmutation/advanced.py debe contener exactamente**:

- Import relativo: `from .basic import lead_to_gold`.
- Import relativo: `from ..potions import healing_potion`.
- `philosophers_stone()` - Devuelve "Philosopher's stone created using [lead_to_gold_result] and [healing_potion_result]".
- `elixir_of_life()` - Devuelve .Elixir of life: eternal youth achieved!".

**alchemy/transmutation/___init___.py debe contener**:

- `from .basic import lead_to_gold, stone_to_gem`
- `from .advanced import philosophers_stone, elixir_of_life`

El archivo `__init__.py` del paquete transmutation expone todas las funciones de transmutación para un acceso sencillo. Esto demuestra la organización a nivel de paquete.

**Ejemplo de salida esperada**:
Prueba tus métodos del flujo:

```
$> python3 ft_pathway_debate.py

=== Pathway Debate Mastery ===

Testing Absolute Imports (from basic.py):
lead_to_gold(): Lead transmuted to gold using Fire element created
stone_to_gem(): Stone transmuted to gem using Earth element created

Testing Relative Imports (from advanced.py):
philosophers_stone(): Philosopher's stone created using Lead transmuted to gold using Fire element
created and Healing potion brewed with Fire element created and Water element created
elixir_of_life(): Elixir of life: eternal youth achieved!

Testing Package Access:
alchemy.transmutation.lead_to_gold(): Lead transmuted to gold using Fire element created
alchemy.transmutation.philosophers_stone(): [same as above]

Both pathways work! Absolute: clear, Relative: concise
```

Tu ft_pathway_debate.py debería demostrar la diferencia entre imports absolutos (ruta completa) e imports relativos (puntos). Muestra cómo ambos métodos acceden a las mismas funciones.

¿Cuándo debería una persona alquimista usar rutas absolutas frente a rutas relativas? ¿Cuáles son los compromisos entre claridad y concisión?

## V.4. Parte IV: Rompiendo la Maldición Circular

### Objetivo

Aprende a identificar y romper la temida Maldición de Dependencias Circulares: cuando los hechizos intentan invocarse mutuamente en un bucle infinito, ¡amenazando con destruir tu laboratorio!

### Archivos que hay que crear para la Parte IV

- `ft_circular_curse.py` - Script de demostración (en la raíz del repositorio).
- `alchemy/grimoire/__init__.py` - Inicializador del paquete grimoire.
- `alchemy/grimoire/spellbook.py` - Registra hechizos y sus efectos.
- `alchemy/grimoire/validator.py` - Valida los ingredientes de los hechizos.

### Instrucciones

Crea un escenario que provocaría imports circulares y después aprende las antiguas técnicas para romper la maldición.

**alchemy/grimoire/___init___.py debe contener**:

- `from .spellbook import record_spell`
- `from .validator import validate_ingredients`

**alchemy/grimoire/validator.py debe contener exactamente**:

- `validate_ingredients(ingredients: str) ->str` - Devuelve "[ingredients] - VALID.º "[ingredients] - INVALID".
- Validación simple: los ingredients que contienen "fire", "water", .ºearth.º .ºair"son válidos.
- Cualquier otro ingredients es inválido.

**alchemy/grimoire/spellbook.py debe contener exactamente**:

- `record_spell(spell_name: str, ingredients: str) ->str` - Registra hechizos después de la validación.
- Debe usar validator.py para comprobar los ingredients antes de registrar (usa importación diferida para evitar la dependencia circular).
- Devuelve "Spell recorded: [spell_name] ([validation_result])"si es válido.
- Devuelve "Spell rejected: [spell_name] ([validation_result])"si es inválido.

**Rompiendo la maldición circular: elige UN método**:

- **Método 1 - Importación Diferida**: importar validator dentro de la función record_spell.
- **Método 2 - Inyección de Dependencias**: pasar la función validator como parámetro.
- **Método 3 - Módulo Compartido**: crear utilidades de validación separadas.

NO crees imports circulares reales en tu código. Demuestra que entiendes el problema implementando uno de los métodos de solución y explicándolo.

**Ejemplo de salida esperada**:
Prueba tus técnicas para romper la maldición:

```
$> python3 ft_circular_curse.py

=== Circular Curse Breaking ===

Testing ingredient validation:
validate_ingredients("fire air"): fire air - VALID
validate_ingredients("dragon scales"): dragon scales - INVALID

Testing spell recording with validation:
record_spell("Fireball", "fire air"): Spell recorded: Fireball (fire air - VALID)
record_spell("Dark Magic", "shadow"): Spell rejected: Dark Magic (shadow - INVALID)

Testing late import technique:
record_spell("Lightning", "air"): Spell recorded: Lightning (air - VALID)

Circular dependency curse avoided using late imports!
All spells processed safely!
```

Tu ft_circular_curse.py debería demostrar cómo evitar imports circulares usando importación diferida (importar dentro de las funciones) o inyección de dependencias. Muestra tanto ejemplos con ingredientes válidos como inválidos.

¿Qué causa la Maldición de Dependencias Circulares y por qué es peligrosa? ¿Qué técnica para romper la maldición es más adecuada en cada situación?

# Capítulo VI

# Instrucciones de Entrega

Entrega tu trabajo en tu repositorio `Git` como de costumbre. Solo se evaluará el contenido dentro de tu repositorio durante la defensa. No dudes en verificar que los nombres de tus archivos sean los correctos.

**Organización de archivos**:
Todos los archivos y directorios deben crearse en la raíz de tu repositorio de Git:

- `ft_sacred_scroll.py` - en la raíz del repositorio.
- `ft_import_transmutation.py` - en la raíz del repositorio.
- `ft_pathway_debate.py` - en la raíz del repositorio.
- `ft_circular_curse.py` - en la raíz del repositorio.
- `alchemy/` - directorio en la raíz del repositorio.
- `alchemy/__init__.py` - dentro del directorio alchemy.
- `alchemy/elements.py` - dentro del directorio alchemy.
- `alchemy/potions.py` - dentro del directorio alchemy.
- `alchemy/transmutation/` - subdirectorio dentro de alchemy.
- `alchemy/transmutation/__init__.py` - dentro del directorio transmutation.
- `alchemy/transmutation/basic.py` - dentro del directorio transmutation.
- `alchemy/transmutation/advanced.py` - dentro del directorio transmutation.
- `alchemy/grimoire/` - subdirectorio dentro de alchemy.
- `alchemy/grimoire/__init__.py` - dentro del directorio grimoire.
- `alchemy/grimoire/spellbook.py` - dentro del directorio grimoire.
- `alchemy/grimoire/validator.py` - dentro del directorio grimoire.

Durante la evaluación, es posible que se te pida explicar los mecanismos de import, demostrar diferentes estilos de import o modificar tu laboratorio alquímico. Asegúrate de entender los cuatro misterios sagrados, no solo la implementación.

Céntrate en escribir un código limpio y bien organizado que demuestre claramente el sistema de imports de Python. Las funciones alquímicas deberían ser simples: la complejidad está en dominar los misterios de los imports.

**RECORDATORIO FINAL**: Esta actividad requiere un dominio sólido de los fundamentos de Python. Si tienes dificultades con la sintaxis básica de Python, functions, classes, listas, diccionarios o gestión de errores, fortalece primero estas habilidades. El dominio de los imports se construye sobre estas bases: intentar esta actividad sin ellas llevará a confusión y frustración.

Alguien que fuera realmente alquimista en Python entendería que una buena organización del código es como un laboratorio bien organizado: todo tiene su lugar y podría encontrar cualquier hechizo rápidamente cuando lo necesitase.